ONE-PAGE SETUP PATH

Luke's Beginner Guide to Codex, Git, and GitHub

The goal is simple: install the tools once, connect your GitHub account, make a real project folder, and use Codex where it can actually read files, edit code, run commands, and push work back to GitHub.

What You Are Setting Up

Codex

The AI coding agent. Use it when you want to build, edit, test, or troubleshoot real files on your machine.

Git + GitHub

Git tracks versions on your computer. **GitHub** stores those versions online so you can recover, share, and publish them.

Local Folder

This is where the project lives while you work. Do not use Dropbox as the main working folder. GitHub is the backup/sync point.

Permissions

Codex can ask before changing things. Start cautious. Give broader permissions only when you trust the task and the folder.

Step 1: Accounts You Need

[ ] A GitHub account you can log into: github.com

[ ] A ChatGPT account with Codex access, or an OpenAI API key if you want usage-based billing.

[ ] A password manager or at least a reliable way to keep your GitHub, ChatGPT, and API passwords/tokens safe.

> **Best beginner choice:** sign into Codex with ChatGPT. Use an API key later for automation or advanced workflows.

Step 2: Install the Basic Tools on Windows

Open **PowerShell** and run these one at a time:

```
winget install Codex -s msstore
winget install --id Git.Git
winget install --id GitHub.cli
winget install --id OpenJS.NodeJS.LTS
winget install --id Python.Python.3.14
```

If Windows asks to approve installs, approve them. After installs finish, close and reopen PowerShell.

Step 3: Sign In

1. Open the Codex app and sign in with your ChatGPT account.
2. In PowerShell, check Codex exists:

```
codex --version
```

3. Connect GitHub in the terminal:

```
gh auth login
```

Choose GitHub.com, browser login, and HTTPS unless you already know you need SSH.

Step 4: Make a Real Project Folder

Use a normal local folder. This keeps Codex, Git, and GitHub working together cleanly.

```
mkdir C:\Users\Luke\Documents\GitHub
cd C:\Users\Luke\Documents\GitHub
mkdir rent-everything
cd rent-everything
git init
```

Open this folder in Codex. This is now the workspace Codex can safely read and edit.

Step 5: Let Codex Prove It Works

In Codex, start with a small test prompt:

```
Create a README.md for a simple app called Rent Everything.
Make a dev branch first. Do not push until I approve.
```

Then check the result:

```
git status
git branch
```

If you see a changed `README.md` and a branch like `dev`, you have the basic loop working.

Step 6: Put It on GitHub

Create the empty repository on GitHub first if permissions block the terminal from creating it. Then connect your local folder:

```
git add .
git commit -m "Start Rent Everything guide project"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/rent-everything.git
git push -u origin main
```

For future work, use a branch:

```
git checkout -b dev
```

The Workflow to Remember

Daily Build Loop

1. Open the project folder in Codex.
2. Tell Codex the exact goal.
3. Let it change files and run checks.
4. Review the diff.
5. Commit and push when it works.

Good First Prompt

```
I want to build [thing].
Use the current folder.
Work on a dev branch.
Explain before installing packages.
Run a basic test or smoke check.
Stop before pushing.
```

> **Do not start with full access.** Use approval prompts until you understand what Codex is changing. Full access is powerful, but a bad prompt or wrong folder can make a mess quickly.

Terms That Were Confusing in the Demo

- **Terminal / PowerShell:** the place where commands run.
- **VS Code:** a good editor for reading files, Markdown, and code.
- **Markdown:** plain text files ending in `.md`; readable by people and easy for AI to use.
- **Branch:** a working copy of the project. Use `dev` for changes before they go live.
- **Pull request:** the GitHub review step before merging branch changes into `main`.
- **Supabase:** a database service. Codex can help write SQL and scripts, but protect API keys and start with manual approval.

When Something Gets Stuck

1. Check whether Codex is waiting for approval.
2. Run `git status` to see what changed.
3. Make sure PowerShell is in the right folder with `pwd`.
4. If GitHub fails, run `gh auth status` and log in again if needed.
5. If the task got too big, start a new Codex thread with a smaller request.

What Success Looks Like

- [ ] Codex opens and is signed in.
- [ ] `codex --version` works in PowerShell.
- [ ] `git --version` works.
- [ ] `gh auth status` says you are logged into GitHub.
- [ ] You have a local project folder under `Documents\GitHub`.
- [ ] Codex can create or edit a file in that folder.
- [ ] You can commit and push that file to GitHub.

Built from Drew and Luke's transcript notes plus current OpenAI Codex documentation. Official references: Codex quickstart, Codex authentication, Codex Windows setup, Codex CLI, and AGENTS.md guidance.